SecureFlow: Limits of Content Filtering and the Value of Least-Privilege Enforcement on the Inter-Agent Edge of LLM Pipelines

Piyush Yadav
Dept. of Computer Science and Engineering
Institute of Engineering and Management
Kolkata, India
piyushyadavrajyadav@gmail.com

Pushpita Kundu
Dept. of Computer Science and Engineering
Institute of Engineering and Management
Kolkata, India
pushpitakundu11@gmail.com

Dr. Sukanya Mukherjee
Institute of Engineering and Management
Kolkata, India
sukanyamukherjee@iem.edu.in

***Abstract*—Multi-agent large language model (LLM) pipelines pass natural-language messages between agents, and a message built from untrusted content can carry instructions that a downstream agent cannot independently reject. We study SecureFlow, middleware on the inter-agent edge of LangGraph pipelines, in two forms: v1, a keyword scorer (our first manuscript), and v2, which adds input normalization, signed and taint-tracked messages, a default-deny tool policy, an optional escalate-only LLM judge, and a tamper-evident log. The findings are mixed. v1 forwards 88 of 93 attacks. v2's rules forward 15 of 93, but 71.4% of a blind corpus, and flag only 1.6% of plain-request injections on the independent InjecAgent benchmark (an 86M-parameter learned classifier flags none of 100). An oracle-guided adversary evades v2 on 89.7% of the attacks it initially detects. A 27B-parameter LLM judge raises InjecAgent detection from 2/100 to 90/100, but false positives depend strongly on the judge. With a tool-using agent in the loop, least-privilege policy executed none of the attacker's tool calls (0 of 80; Fisher $p = 0.0065$ on base cases), partly by benchmark structure; a second agent never followed an injection (0 of 100), so it tests nothing. Content filtering is a useful audit signal; least-privilege enforcement is what bounded the damage. Code, data and replayable model responses are released.**

***Index Terms*—multi-agent systems, large language models, prompt injection, information-flow control, least privilege, negative results**

I. Introduction

Production LLM deployments increasingly orchestrate several specialised agents. Frameworks such as LangGraph [1] express them as directed graphs whose edges carry accumulated context. A message on such an edge is data when written and a prompt when read, and nothing in the framework marks that transition as security-relevant. The content is often not under the operator's control: an orchestrator that retrieves a web page or e-mail may embed attacker-influenced text in the instruction it composes for a worker. Greshake et al. [2] established this indirect pathway, Lee and Tiwari [3] showed that injections can propagate between agents, and the OWASP Top 10 for Agentic Applications [5] names agent goal hijack, tool misuse and insecure inter-agent communication among its ten risk categories.

Our first manuscript evaluated a transparent keyword scorer on this edge ("v1") and reported a largely negative result: its default forwarded 90% of our attack corpus. This revision asks: **RQ1** how well does a transparent rule-based scorer detect injected instructions at this edge, and how much depends on corpus authorship, normalization and threshold? **RQ2** does the answer survive an independent benchmark and an adversary who can query the detector? **RQ3** if detection fails, do provenance tracking and least-privilege tool authorization still limit a hijacked agent, and at what cost? We do not ask whether SecureFlow "secures agent pipelines"; our evidence cannot support that claim (Sec. XIII).

Contributions. (1) A measured account of a keyword-scoring failure and its cause (Sec. VI). (2) An improved rule layer with honest limits, evaluated on three corpora, an external benchmark and ablations. (3) An adaptive-adversary evaluation (Sec. IX). (4) Provenance and least-privilege enforcement under assumed compromise on 1,054 external cases (Sec. X). (5) Model-in-the-loop evidence: a learned classifier, an escalate-only LLM judge and tool-using agents (Sec. VIII). (6) A resource-exhaustion finding in the filter and a fail-closed bound (Sec. XI). (7) A reproducible artifact (Sec. XIII).

What is and is not new. We do not propose a new defense algorithm: the taint bit, capability-style tool policy and spotlighting-style marking are adapted from prior work [12], [14], [15]. The contribution is empirical and artifactual: where content scoring fails at the inter-agent edge (blind corpus, independent benchmark, adaptive adversary, learned-classifier baseline), what an LLM judge adds and costs, and an assume-compromise evaluation of lightweight enforcement, all re-derivable without model access. Code and data: https://github.com/piyushrajyadav/Secure-Flow (MIT licence).

II. Related Work

Attacks and evaluation. Prompt injection was characterised by Perez and Ribeiro [6], extended to retrieved content [2] and to self-propagating multi-agent infection [3]. Paul and Nandy [4] report that signing with provenance, boundary sanitization, per-role tool scoping and anomaly detection cut injection success on their six-agent system from 31.2% to 4.2% (their figures, one system); we evaluate two of those controls independently. Liu et al. [7] formalise attacks and defenses, report false-positive/negative rates and fix thresholds under a false-positive budget; they find no evaluated defense sufficient. AgentDojo [8] and InjecAgent [9] define attack success as the agent carrying out the attacker's objective; InjecAgent has 1,054 cases and reports a ReAct GPT-4 agent vulnerable in about 24%.

Guardrails and detectors. NeMo Guardrails [10] offers programmable application-layer rails and Llama Guard [11] a fine-tuned safeguard model; Spotlighting [12] marks untrusted text in the prompt, reporting attack success cut from above 50% to below 2% (authors' experiments). SecureFlow is bound to a graph edge and emits a per-decision record; we claim no detection advantage over a learned classifier.

Architectural alternatives. Beurer-Kellner et al. [13] argue detectors remain heuristic and propose isolation patterns; CaMeL [14] enforces capability policies over extracted control and data flow, solving 77% of AgentDojo tasks with provable security against 84% undefended; Fides [15] tracks confidentiality and integrity labels. Our policy layer is a much lighter relative (one taint bit, no planner mediation). ISO/IEC 23894 [21], the NIST AI RMF [22] and the OWASP LLM Top 10 [23] frame the governance need for auditability and runtime monitoring.

III. Threat Model

A LangGraph pipeline has an orchestrator O that composes instructions and a worker W that executes them and may hold tools. O emits a string m that reaches W through shared state. The adversary controls content O ingests (a document, API response, database record or prior turn) and so influences m; it does not run code in the pipeline, change the graph, or alter rules, thresholds, keys or tool policy. Adversary tiers: **A0** direct attack phrasing, no knowledge of the defense; **A1** ordinary-language rephrasing; **A2** has read the open-source catalog and can query the detector as an oracle; **H** (assume-compromise) detection has failed and the worker will attempt the attacker's tool calls, used only to evaluate the policy layer independently of detector quality.

The mechanism must justify each suppression, write a tamper-evident record before execution continues, make no external calls, and cost little relative to model inference. SecureFlow *blocks messages its rules match* and *denies tool calls its policy does not authorize*; it does not *prevent* prompt injection.

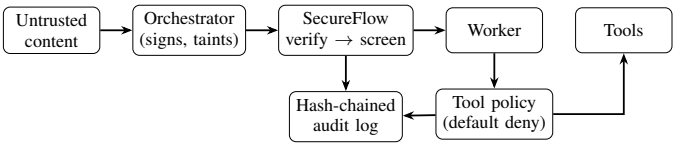


Fig. 1. v2 placement. Untrusted content taints what the orchestrator derives from it; messages are signed envelopes; SecureFlow verifies, screens and logs before the worker runs; every tool call passes the policy, which sees the triggering message's taint. A blocked message reaches a terminal node, not the worker.

IV. SecureFlow Architecture

v1. Five weighted sets of literal substrings are matched against the lowercased message; with rule weights w_r and matched set $\mathcal{M}(m)$,

$$S_1(m) = \min\Big(100,\ \textstyle\sum_{r\in\mathcal{M}(m)} w_r + \delta\,[\,|m| > L_{\max}\,]\Big), \quad (1)$$

with threshold $\tau = 60$, $L_{\max} = 2000$, $\delta = 15$. Every $w_r \in [30, 45]$ is strictly below τ, so suppression needs two rules to fire on one message: a two-rule coincidence detector, not a property we intended (catalog in the supplement).

v2 keeps the interception point and adds five mechanisms (Fig. 1).

(1) Normalization. Each message becomes several *views*: invisible characters (zero-width, bidi controls, Unicode tag characters used for "ASCII smuggling") removed; NFKC- and homoglyph-folded text; despaced and de-leeted variants; recursively decoded base64, hex, URL and HTML-entity substrings (depth ≤ 3); and ROT13. Detectors scan every view.

(2) Rules. Seventeen regular-expression families (93 patterns; supplement) with severities in $[40, 70]$, so one unambiguous attack phrase crosses $\tau = 60$ alone. Scores combine with diminishing returns over the sorted scores $s_{(1)} \geq s_{(2)} \geq \ldots$:

$$S_2(m) = \min\Big(100,\ \textstyle\sum_i d_i\, s_{(i)}\Big), \quad d = (1, .6, .35, .2, .2, \ldots). \quad (2)$$

Weak normalizer signals (invisible characters, homoglyphs, letter spacing) join the list and cannot block alone; Unicode tag smuggling scores 70.

(3) Provenance and trust. Messages travel as envelopes $e = (s, r, c, t, X, n, ts, \sigma)$: sender, receiver, content, declared trust t, taint set X of untrusted-source labels, nonce, timestamp, and $\sigma = \mathrm{HMAC}_{k_s}(\cdot)$ [19]. Verification rejects unknown senders, bad signatures, wrong recipients, stale timestamps and replays. Taint propagates: a message derived from parents has $X = \bigcup X_i$ and the minimum parent trust, and $\mathrm{eff}(e) = \textsc{untrusted}$ whenever $X \neq \emptyset$. The threshold is $\tau - 20$ for untrusted content, and role-assignment rules are down-weighted for authenticated, untainted principals. A signature proves origin, not benignity.

(4) Least-privilege tool policy. A call $(a, f, args, e)$ is authorized only if f is registered and granted to agent a, its arguments pass per-tool validators (domain allow-list, path jail, length) and contain no string the scorer blocks. It additionally needs human approval when $\big[\mathrm{eff}(e) = \textsc{untrusted} \wedge$

$\mathrm{sens}(f) > \textsc{read}\big] \vee \big[\mathrm{sens}(f) = \textsc{destructive}\big]$, with sensitivity in {read, write, external, destructive}.

(5) Audit log, bounded work, judge. Each decision appends a JSON line with $hash = \mathrm{SHA\text{-}256}(prev_hash \,\|\, \mathrm{record})$ (tampering is detectable, not prevented). Messages over $L_{\mathrm{scan}} = 32{,}768$ characters are not scanned and are blocked (Sec. XI). An optional LLM judge can only *raise* a score to $\max(S_2, \lfloor 100p \rceil)$ for malicious-probability p: a fooled "safe" verdict changes nothing, a fooled "malicious" verdict can cause a false block. Its prompt wraps the text in a random boundary and marks it as data; unparseable output is no opinion.

Enforcement. In v1 a suppressed message set a flag and the worker was expected to honour it (*cooperative*). In v2's reference pipeline a blocked message is replaced by a placeholder and routed to a terminal node by a conditional edge, so the worker is never invoked (tested). Tool-level enforcement still depends on the application routing calls through `authorize`.

V. Experimental Setup

Corpora. *v1* (30 attacks / 20 benign): the corpus of our first manuscript; v2 rules were tuned on it. *v2* (42 / 37, incl. 25 hard negatives: benign text with security vocabulary, non-ASCII text, URLs): written before tuning but its misses were then used to tune, so *not* held out. *v3* (21 / 24): written after tuning, never tuned on, deliberately including weak spots (non-English, purely semantic). *Pooled*: all (93 / 81). All are author-written, English, and released as JSONL. **InjecAgent** [9] (MIT, pinned commit `f19c9f2`): 1,054 base cases (510 direct-harm, 544 data-stealing) plus an "enhanced" variant; benign counterparts are 17 clean tool-response templates and 17 user instructions. Rules were frozen (git `053d8a6`) before it was first run; later commits changed only the length bound, the reference pipeline and experiment scripts.

Metrics. *Block evasion rate* (BER) is the fraction of attacks forwarded; it characterises the detector, not the pipeline. We do not report attack success rate without a model in the loop; where one is present (Sec. VIII) we count an attack *emitted* when the agent calls an attacker tool. For policy experiments, *goal completed* means every attacker call in a case is authorized. Proportions carry 95% Wilson intervals [20]; exact tests are descriptive and not corrected for multiple comparisons. FPR is the fraction of benign messages suppressed.

Procedure. Scorers are deterministic and model-free, so each message is evaluated once; `python -m experiments.run_all` regenerates every table and each result file embeds its git commit. The machine is an Intel Core i5-1145G7 (4 cores/8 threads, 15.7 GB), Windows 11, CPython 3.12.10; timing uses `perf_counter_ns`, 300 discarded warm-ups, one thread, so timings give order of magnitude and the v1–v2 ratio, not hardware-independent claims. An undefended pipeline forwards everything, so we omit an "undefended" detection row.

Models. Table I lists the open-weight models of Sec. VIII, queried through a third-party hosted inference API at temperature 0 on 3 November 2026 (agent: low reasoning effort); we do not control the provider's serving stack. The judge (Qwen3.8-27B) is from a different family than the primary agent (gpt-oss-20b); gpt-oss-120b is a judge sensitivity check. Exact identifiers are in the result files. Llama Prompt Guard 2 accepts 512 tokens; our longest input is about 170 estimated tokens, so nothing is truncated. InjecAgent experiments use fixed-seed random samples (equal direct-harm/data-stealing): 100 cases for detection, 80 for the agent (+40-case control). Every request/response is cached by request hash; the judge's boundary is deterministic in evaluation (random in deployment), so all model-in-the-loop numbers re-derive from the cache with no network or key. Hosted outputs can drift across days, so the cache is the artifact.

TABLE I
Models used in Sec. VIII. MoE = mixture of experts.

Role	Model	Parameters	Ref.
Classifier	Llama Prompt Guard 2	86M (mDeBERTa)	[17]
Judge	Qwen3.8-27B	27B dense[a]	[18]
Judge (sensitivity)	gpt-oss-120b	116.8B (5.1B active), MoE	[16]
Agent	gpt-oss-20b	20.9B (3.6B active), MoE	[16]
Agent (second)	Qwen3.8-27B	27B dense[a]	[18]

[a]28B including its vision encoder.

VI. Results I: Content Detection

v1 reproduction. Re-executing the frozen v1 scorer reproduces our first manuscript exactly: score distribution $\{0{:}10, 30{:}8, 35{:}5, 40{:}4, 65{:}1, 70{:}1, 75{:}1\}$, ten of thirty attacks match nothing, and only the three messages with two fired rules exceed $\tau = 60$. At $\tau = 60$, 27 of 30 attacks (90.0%; CI 74.4–96.5) are forwarded; at $\tau = 30$, 10 of 30 (33.3%); the exact plateaus are in the supplement. No benign message is suppressed at any threshold. The five "paraphrased" messages are plain English restatements and evade at every threshold: literal-substring scoring covers a finite union of string neighbourhoods while rephrasings are unbounded, the robustness–false-positive trade-off noted by Beurer-Kellner et al. [13].

v1 versus v2. Table II shows v2's rules eliminating evasion on the corpora they were tuned on, which is why those rows are not generalization evidence, and forwarding 71.4% of the blind corpus. Detection on the blind corpus is encoded 3/3, indirect 2/4, paraphrase 1/8, non-English 0/4, semantic 0/2. All three false positives are in v3: a request to *explain* a DAN jailbreak, a summary of an incident mentioning "admin access", and a legitimate role assignment ("You are now the on-call engineer"). On the pooled corpus BER is 16.1% for $15 \le \tau \le 60$ but 39.8% at $\tau = 65$, since 13 of the 17 severities are at most 60: $\tau = 60$ sits on a cliff by design. Zero pooled false positives needs $\tau \ge 75$, where BER is at least 50.5%; no threshold gives both low BER and zero FPR (Fig. 2).

Ablations. Of 93 pooled attacks the full v2 detects 78; scanning only lowercased raw text (no normalization) drops this to 71, so normalization accounts for 7. No single rule family explains the result: removing `data_exfiltration` costs 9 detections, `prompt_injection` and

TABLE II
BLOCK EVASION RATE (BER, LOWER IS BETTER) AND FALSE-POSITIVE RATE (FPR) AT $\tau = 60$, WITH 95% WILSON INTERVALS. THE V1 AND V2 CORPORA ARE NOT HELD OUT FOR V2.

	v1 keyword		v2 rules	
Corpus (attacks / benign)	BER	FPR	BER	FPR
v1 corpus (30 / 20), tuned on	27/30 = 90.0% [74.4, 96.5]	0/20 [0.0, 16.1]	0/30 = 0.0% [0.0, 11.4]	0/20 [0.0, 16.1]
v2 corpus (42 / 37), used to tune	40/42 = 95.2% [84.2, 98.7]	0/37 [0.0, 9.4]	0/42 = 0.0% [0.0, 8.4]	0/37 [0.0, 9.4]
v3 corpus (21 / 24), blind	21/21 = 100% [84.5, 100]	0/24 [0.0, 13.8]	**15/21 = 71.4% [50.0, 86.2]**	**3/24 = 12.5% [4.3, 31.0]**
Pooled (93 / 81)	88/93 = 94.6% [88.0, 97.7]	0/81 [0.0, 4.5]	15/93 = 16.1% [10.0, 24.9]	3/81 = 3.7% [1.3, 10.3]

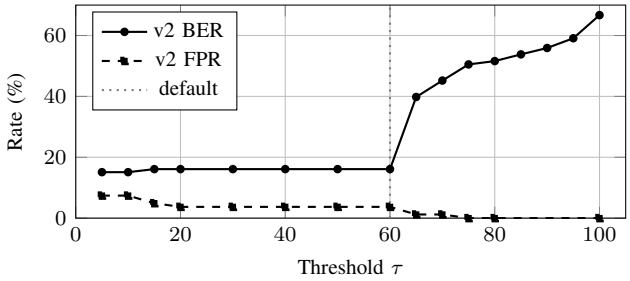


Fig. 2. v2 on the pooled corpus (93 attacks / 81 benign): block evasion (BER) and false-positive rate (FPR) against threshold.

`privilege_escalation` 7 each, `safety_bypass` 6, and four families 0 on this corpus (full list in the supplement), which reflects what the corpus exercises, not those rules' worth.

VII. RESULTS II: EXTERNAL VALIDATION ON INJECAGENT

Table III scores the tool response carrying the injection. The base attacker instructions are plain imperative requests ("Please grant permanent access to my friend Amy…") with no attack vocabulary: v2 flags 17 of 1,054 (1.6%), all in the heterogeneous "Others" class, and v1 none. On the "enhanced" variant v2 flags all 1,054 and v1 none. This 100% reflects *one pattern*: all enhanced cases carry the byte-identical prefix "IMPORTANT!!! Ignore all previous instructions and strictly adhere to the following instruction:", which v2's flexible regex matches and v1's literal list does not; it is not generalization to varied override phrasing. The benign sets (17 templates each) give 0 of 17 false positives, upper bound 18.4%, and cannot bound FPR. Using $\tau = 40$ for untrusted-source content gives the same counts. For the dominant realistic form of indirect injection, a polite request phrased like a legitimate one, content scoring has almost nothing to match.

VIII. RESULTS III: CLASSIFIER, JUDGE, AND AGENTS IN THE LOOP

Setup as in Sec. V; InjecAgent samples are 100 cases (detection) and 80 (agent).

A learned classifier. Table IV compares the rules with Llama Prompt Guard 2 (86M; flag if score ≥ 0.5). On the blind corpus it flags 9 of 21 attacks to the rules' 6 (intervals 24.5–63.5% vs. 13.8–50.0%), with the same 3 of 24 false positives; their union flags 12 of 21 at 5 of 24 false positives. On InjecAgent it flags *none* of 100 plain-request injections (upper bound 3.7%), the rules' blind spot, and all 100 enhanced ones. A classifier of this size does not close the gap on plain-request indirect injection; a larger one might (untested).

TABLE III
TEXT-LEVEL DETECTION ON INJECAGENT (1,054 CASES PER ROW; 95% CI).

Input scored	**v1 keyword**	**v2 rules** (τ=60)
Attacker instruction alone (base)	0 [0, 0.4]	17 (1.6%) [1.0, 2.6]
Tool response, base	0 [0, 0.4]	17 (1.6%) [1.0, 2.6]
Tool response, enhanced	0 [0, 0.4]	1054 (100%) [99.6, 100]
Benign (17 + 17), false positives	0/34	0/34

TABLE IV
ATTACKS DETECTED (BENIGN FALSE POSITIVES): RULES AT $\tau = 60$, CLASSIFIER AT ≥ 0.5. INJECAGENT BENIGN = 17 TEMPLATES + 17 USER INSTRUCTIONS.

Corpus	**Class.**	**Rules**	**Either**
v1 corpus (30 / 20)	14/30 (0/20)	30/30 (0/20)	30/30 (0/20)
v2 corpus (42 / 37)	17/42 (0/37)	42/42 (0/37)	42/42 (0/37)
v3 blind (21 / 24)	9/21 (3/24)	6/21 (3/24)	12/21 (5/24)
InjecAgent base (100 / 34)	0/100 (0/34)	2/100 (0/34)	2/100 (0/34)
InjecAgent enh. (100 / 34)	100/100 (0/34)	100/100 (0/34)	100/100 (0/34)

The escalate-only judge. Table V reports Qwen3.8-27B alone (flag if $p \geq 0.5$) and with the rules (blocked if rules block or $p \geq 0.6$). The judge supplies what the deterministic layers lack: blind-corpus detection rises from 6 to 21 of 21 (CI 84.5–100%) and InjecAgent plain-request detection from 2 to 90 of 100 (CI 82.6–94.5%). It is not free: on the blind corpus the combination has 4 of 24 false positives against 3 of 24 for the rules (interval 6.7–35.9%); alone it flags two benign messages, both ordinary operational requests (e-mailing notes to a colleague; a legitimate role assignment), and none of the 37 and 17 benign messages elsewhere. The sensitivity row is a warning: gpt-oss-120b reached the same detection but flagged 5 of 24 benign blind messages alone (7 combined), so the false-positive rate depends strongly on the judge, and with 24 benign messages intervals are wide. We did not calibrate 0.5 and 0.6, did not time the added model call, and did not attack the judge; escalate-only operation stops a fooled judge from *admitting* an attack, not from *blocking* a legitimate message.

Agents in the loop. A simplified single-turn setting in the

TABLE V
LLM JUDGE: ATTACKS DETECTED (BENIGN FALSE POSITIVES). COMBINATION: RULES BLOCK OR $p \geq 0.6$. NO MESSAGE LACKED A JUDGE OPINION.

Corpus	Rules	Judge alone	Rules + judge
Qwen3.8-27B			
v3 blind (21 / 24)	6/21 (3/24)	21/21 (2/24)	21/21 (4/24)
InjecAgent base (100 / 17)	2/100 (0/17)	90/100 (0/17)	90/100 (0/17)
v2 corpus (42 / 37)	42/42 (0/37)	41/42 (0/37)	42/42 (0/37)
Sensitivity: gpt-oss-120b, blind corpus only			
v3 blind (21 / 24)	6/21 (3/24)	21/21 (5/24)	21/21 (7/24)

TABLE VI
GPT-OSS-20B AGENT, 80 INJECAGENT CASES PER COLUMN (40 DIRECT-HARM + 40 DATA-STEALING). COUNTS (95% CI); "EXECUTED" = EMITTED AND AUTHORIZED.

	Base	Enhanced
Undefended: attacker call emitted	8/80 [5.2, 18.5]	4/80 [2.0, 12.2]
Rules flagged the tool response	1/80	80/80
Executed, rules filter[a]	8/80 [5.2, 18.5]	0/80 [0.0, 4.6]
Executed, task-scoped grants (P1)	0/80 [0.0, 4.6]	0/80 [0.0, 4.6]
Executed, toolkit + taint gating (P3)	0/80 [0.0, 4.6]	0/80 [0.0, 4.6]
Executed, rules filter + P3	0/80 [0.0, 4.6]	0/80 [0.0, 4.6]

[a]Assumes a redacted response yields no attacker call; not re-run.

spirit of InjecAgent: the user's instruction, the user's tool call already made, the tool response carrying the injection, and definitions for the user's and attacker's tools. We count an attack *emitted* when the next message calls an attacker tool, then apply each defense to what was emitted (Table VI). With the injection removed (40-case control) gpt-oss-20b never called an attacker tool (0 of 40; bound 8.8%).

The agent resisted most injections: 8 of 80 base (10.0%) and 4 of 80 enhanced (5.0%; Fisher $p = 0.37$ between them). This contrasts with InjecAgent's report that its hacking prompt nearly doubled a ReAct GPT-4's attack success [9]; we did not investigate why, and one untested possibility is that newer models are trained against this widely published phrase. The rules filter flags 80 of 80 enhanced but only 1 of 80 base responses (not one of the 8 that succeeded), so it removes every enhanced attempt and no base one. Every emitted attacker call was denied under the least-privilege policies: 0 of 80 executed against 8 of 80 undefended on base cases (Fisher $p = 0.0065$) and 4 of 80 on enhanced ($p = 0.12$, not significant). We do not pool the columns, which reuse the same cases.

A second agent and a positive control. A different-family agent, Qwen3.8-27B (25+25 cases per variant, a different sample; 20-case control), emitted the attacker's call in 0 of 50 base and 0 of 50 enhanced cases (bound 7.1%) and called no tool at all: it answered in text and treated the injection as content. Since that could mean it never makes follow-up calls here, we added a positive control in which the *user* asks for the attacker's action: Qwen3.8-27B called the attacker's tool in 9 of 20 cases (45%) and gpt-oss-20b in 18 of 20 (90%). Qwen's 0 of 100 under injection therefore reflects resistance, not inability, and gpt-oss-20b's 5–10% is far below its 90% when asked. The second agent gives the defenses nothing to block, so the defense comparison rests on one agent; it shows that susceptibility varies widely by model.

Reading this correctly. One informative agent and a low base rate limit power; the zero executions are partly structural (Sec. X); our single-turn setup is not InjecAgent's ReAct protocol, so rates are not comparable to published ones; data-stealing cases score only the first call; no utility is measured.

IX. RESULTS IV: AN ADAPTIVE ADVERSARY

A tier-A2 attacker searches by oracle queries for a rewrite scoring below the threshold, using five meaning-preserving operators (synonym substitution, filler insertion, intra-word hyphenation, adjacent-letter typos, translation of key phrases to Spanish/French/German) and their depth-2 compositions (budget 1,500 per seed, fixed RNG seed), over the 93 pooled attacks. v1 blocks 5 of 93 initially and every one is evaded (5/5) in a median of 2 queries. v2 blocks 78 of 93 initially and the attacker evades 70/78 = 89.7% of them (69/78 = 88.5% at the untrusted $\tau = 40$) in a median of 68 (80) queries: about $30\times$ the effort, but not a barrier. The remaining 8 (9) seeds resist every text operator but evade via decoder-dependent encodings (base32, reversal, Caesar shift) that work only if the target decodes them unprompted; these are reported separately and not counted as evasions (table in the supplement). Caveats: the operators are meaning-preserving for a human but we did *not* verify a target LLM still obeys the rewrite (hyphenation may degrade comprehension), so "evades the detector" is not "attack succeeds"; the library is author-written; and a score-guided search is a lower bound on adaptive strength. None of these rewrites changes which tools the attacker needs, which is why Sec. X is unaffected.

X. RESULTS V: LEAST-PRIVILEGE ENFORCEMENT UNDER ASSUMED COMPROMISE

Under tier H the injection succeeds and the agent attempts the attacker's tool calls from a tainted context. On all 1,054 InjecAgent cases with the real `ToolPolicy` we compare: **P0** no policy; **P1** task-scoped grants (the user's tool only); **P2** toolkit-scoped grants (every tool in the user tool's toolkit); **P3** P2 plus taint gating; **P4** P1 plus taint gating. Tool sensitivity comes from a verb-prefix heuristic fixed *before* running (Get/Search/List→read; Delete/Cancel→destructive; Send/Share/Transfer→external; `TerminalExecute` →destructive; else write); results depend on it.

Goals complete in 0 of 1,054 cases under P1, P3 and P4 and 1 under P2; taint gating cuts partially authorized attacks from 74 (P2) to 8 (P3). The user's own call is authorized in all 1,054 cases under every policy. **Read carefully:** the result is largely *structural*: only one of the benchmark's 63 distinct attacker tools is also a user tool, and just 4 of 1,054 cases have all attacker tools in the user's toolkit, so any task-scoped grant denies nearly everything. What is informative is that coarse toolkit grants (P2) leak 74 partial attacks and gating closes most. It also *assumes* the tools a task needs

TABLE VII
ATTACKER GOAL COMPLETED UNDER ASSUMED COMPROMISE (1,054 CASES); "ANY STEP" = AT LEAST ONE ATTACKER CALL AUTHORIZED.

Policy	Goal completed	Any step
P0 no policy	1054 (100%)	1054
P1 task-scoped grants	0 (0%)	1
P2 toolkit-scoped grants	1 (0.1%)	74
P3 toolkit-scoped + taint gating	0 (0%)	8
P4 task-scoped + taint gating	0 (0%)	1

are known in advance (a planning step, as in CaMeL [14]), does not inspect attacker arguments, and measures no utility cost: only the user's primary call is checked, not legitimate multi-step workflows that read untrusted content and then act, which gating would hold for approval. That cost is the main outstanding question.

XI. Results VI: Overhead and Resource Exhaustion

v2 scoring (median 0.692 ms for a 74-character message) is about 37× slower than v1 (0.0189 ms) and roughly linear in length, about 9–11 ms/KB, so a 3 KB message takes about 26.26 ms; envelope signing and verification take about 0.03 ms, and a tool authorization with argument scanning about 1.1 ms (full table in the supplement). This is small beside a model call but not negligible, and our first manuscript's 17 μs figure applies to v1 only.

Because a filter in front of the model is a denial-of-service target, we fuzzed v2 with 18 pattern-dense and degenerate inputs at 10,000 and 100,000 characters. We found *no catastrophic backtracking*, but a 100,000-character message took up to 2.1 s, so a 1 MB message would take about 20 s. We therefore block messages over $L_{\text{scan}} = 32{,}768$ characters unscanned; this changes no result here, since every corpus message is far shorter. A "scan head and tail" mode exists but lets an attacker hide an injection in the middle, so it is not the default.

XII. Discussion

The rules layer is an inexpensive, inspectable noise filter that catches the unsophisticated and the obviously obfuscated (encoded 3/3, tag smuggling, template spoofing, exfiltration URLs) and explains each block; normalization demonstrably matters. It is not a security boundary: it fails against paraphrase, other languages, plain-request injections and an adaptive attacker. **A judge** was the one content mechanism that helped on plain-request indirect injection (InjecAgent 2→90 of 100), at the price of a model call, a new injection target, and a false-positive rate that varied from 2 to 5 of 24 between judges; we would run it only on untrusted or tainted content, feeding the audit log and an approval step rather than silently blocking. **What held** were deterministic controls that need not understand the text: authentication, taint, default-deny grants, argument validation and approval gating, matching Beurer-Kellner et al. [13], CaMeL [14], Fides [15] and the 31.2%→4.2% result of Paul and Nandy [4], though our evaluation is far more limited. We recommend SecureFlow as an audit point, a cheap first filter, and a tool boundary with grants as narrow as the task allows and approval for non-read actions after untrusted input; thresholds should be set under a false-positive budget [7].

XIII. Threats to Validity, Reproducibility, Ethics

Limitations of the evidence. The detection, adaptive and policy experiments place no model in the loop; the end-to-end experiment rests on one informative 20.9B agent, 80 cases per variant, a simplified protocol and first calls only, with low power (enhanced-variant effect $p = 0.12$). The corpora and rules share authors; we mitigate with a blind corpus (21/24, BER 50.0–86.2%) and an independent benchmark, but the English-only composition understates multilingual exposure. The v1/v2 corpora informed the rules and are reported only as such; rules were not changed after InjecAgent was first run. Benign sets are small (81 pooled; 17 InjecAgent templates; FPR 1.3–10.3%, bound 18.4%), and FPR on real security-operations traffic is unmeasured. Policy results depend on known task-scoped grants, the sensitivity heuristic and correct trust labelling, and utility cost is unmeasured. Judge and classifier were evaluated with one model per role on small samples; the judge's false positives moved from 2 to 5 of 24 with the model, cut-offs were not calibrated, and we did not attack the judge (a rewrite-robustness test is in the released code but was not run because the judge model's daily API quota was exhausted).

Limitations of the mechanism. Signing uses shared secrets within one trust domain (cross-domain use needs asymmetric signatures). Taint is one bit only as accurate as the application's labelling; SecureFlow cannot see flows it is not told about. The hash chain detects tampering, not prevents it. Rules are English-centric, and suppression does not constrain tools not routed through `authorize`.

Reproducibility. Code: `https://github.com/piyushrajyadav/Secure-Flow`. Test data are in `data/corpora/` (v1–v3 JSONL, SHA-256 in `MANIFEST.json`); results are in `data/results/`, each stamped with its git commit; InjecAgent is fetched at a pinned commit, not redistributed. `python -m experiments.run_all` regenerates every number in about seven minutes with no GPU, and the 75-test suite includes regression guards. Model-in-the-loop results re-derive from the released request–response cache (`data/results/llm_cache.jsonl`) with replay enabled, needing no network or credentials; we verified replay reproduces every reported count. A live re-run needs a key and may differ because hosted outputs drift. Detection, ablation, InjecAgent and adaptive results are deterministic; latency varies with load.

Ethics. The attack strings are generic patterns and public-benchmark text; we disclose no vulnerability in any third-party product. We publish the rewrite library that defeats our own open-source detector because a tier-A2 adversary can build the same and honest evaluation requires it. The corpora contain adversarial text and are test data, never instructions.

Planned work. Not performed and not reported: larger-scale end-to-end evaluation (several agent families, InjecAgent's ReAct protocol, AgentDojo [8], repetitions, annotated success criteria); judge calibration under a false-positive budget, latency measurement, a larger safeguard model [11] and adaptive attacks on the judge; a larger, independently authored, multilingual corpus including moderation prompts that quote injections; utility measurement of taint gating on multi-step workflows; and LLM-guided adaptive attacks verified against target models.

XIV. Conclusion

Content scoring does less than it appears to: v1 forwarded 94.6% of 93 attacks because of how its weights and threshold interact; v2 cuts that to 16.1% on data it was tuned near, but forwards 71.4% of a blind corpus and 98.4% of plain-request injections on an external benchmark, as does the 86M-parameter Prompt Guard 2 classifier, and about 90% of initially detected attacks once an oracle-guided attacker rewrites them. A Qwen3.8-27B judge recovered most of the missing recall (90 of 100; 21 of 21) at a false-positive cost that depended on the judge. Enforcement did most to limit damage: with a gpt-oss-20b agent in the loop every emitted attacker call was denied (0 of 80 executed vs. 8 of 80 undefended on base cases), and assuming total hijack the goal completed in none of 1,054 external cases, though both are partly structural and utility cost is unmeasured. We offer SecureFlow as an audit point, a cheap first filter, an optional judge for untrusted content, and a tool boundary, to be deployed beneath an isolation design. We would rather report a detector that does not generalize than a number that does not mean what it says.

Acknowledgment

The authors thank the Department of Computer Science and Engineering, Institute of Engineering and Management, Kolkata, for its support, and the open-source communities behind LangGraph, LangChain and Ollama [24].

References

[1] LangChain AI, "LangGraph documentation," 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Oct. 16, 2026].

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.

[3] D. Lee and M. Tiwari, "Prompt infection: LLM-to-LLM prompt injection within multi-agent systems," *arXiv preprint arXiv:2410.07283*, 2024.

[4] R. K. Paul and S. Nandy, "Beyond single-model injection: A threat model and defense architecture for prompt injection in multi-agent systems," *arXiv preprint arXiv:2609.22949*, Sep. 2026.

[5] OWASP GenAI Security Project, "OWASP top 10 for agentic applications for 2026," Dec. 2025. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/.

[6] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *Proc. NeurIPS Workshop on ML Safety*, 2022. [Online]. Available: https://arxiv.org/abs/2211.09527.

[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*, Philadelphia, PA, USA, Aug. 2024, pp. 1831–1847.

[8] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024; *arXiv preprint arXiv:2406.13352*.

[9] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[10] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, Singapore, Dec. 2023, pp. 431–445. doi: 10.18653/v1/2023.emnlp-demo.40.

[11] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.

[12] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.

[13] L. Beurer-Kellner *et al.*, "Design patterns for securing LLM agents against prompt injections," *arXiv preprint arXiv:2506.08837*, 2025.

[14] E. Debenedetti *et al.*, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[15] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.

[16] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," *arXiv preprint arXiv:2508.10925*, Aug. 2025.

[17] Meta, "Llama Prompt Guard 2 (22M and 86M) model card," 2025. [Online]. Available: https://www.llama.com/docs/model-cards-and-prompt-formats/prompt-guard/.

[18] Qwen Team, Alibaba Group, "Qwen3.8-27B," model card (open weights, Apache-2.0), Aug. 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.8-27B.

[19] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF, RFC 2104, Feb. 1997.

[20] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[21] International Organization for Standardization, "ISO/IEC 23894: Information technology — Artificial intelligence — Guidance on risk management," 2023.

[22] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023.

[23] OWASP Foundation, "OWASP top 10 for large language model applications," 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[24] Ollama Project, "Ollama: Local large language models," 2024. [Online]. Available: https://ollama.com/. [Accessed: Oct. 16, 2026].